

12강. SAGA 및 트랜잭션 처리



이영곤



실습 목표

- ✓ 주문(Order) → 재고(Stock) → 결제(Payment) → 배송(Delivery)
- ✓ Kafka 이벤트 체인으로 연결
 - 예: OrderCreated
 - StockReserved
 - PaymentApproved
 - DeliveryStarted
- ✓ 실패 시에는 보상 이벤트
 - StockReserveFailed
 - PaymentFailed
 - OrderCancelled
 - StockReleased

실습 조건

- ✓ 각 서비스는 자기 DB/상태를 가짐
- ✓ 다른 서비스는 직접 호출하지 않음
- ✓ 상태 변경 결과를 이벤트로 발행
- ✓ 다음 서비스는 해당 이벤트를 구독
- ✓ 실패하면 보상 이벤트를 발행

실습 인프라

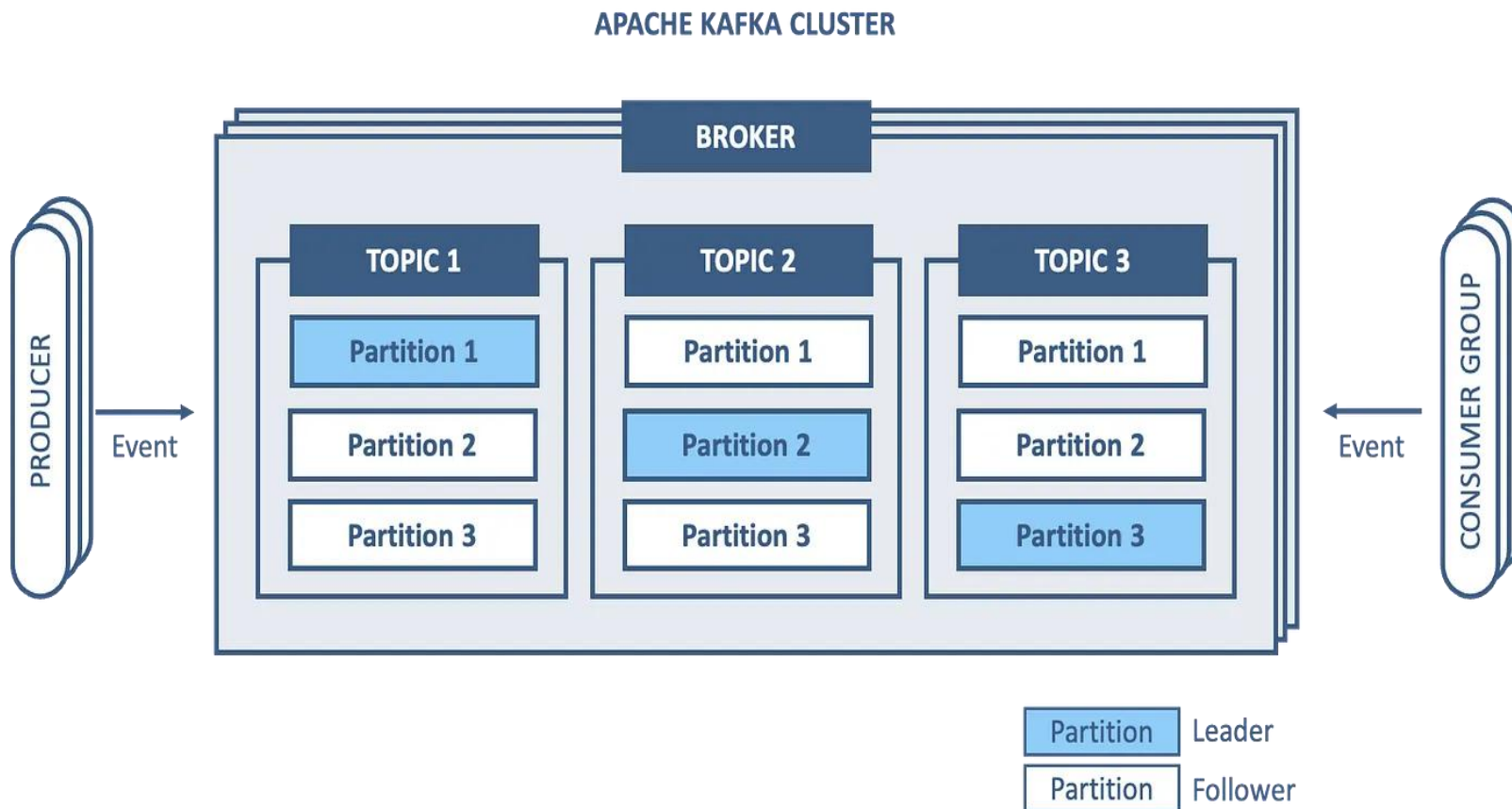
- ✓ Docker Desktop Kubernetes
- ✓ Strimzi Operator
- ✓ Kafka single-node cluster
- ✓ Kafka UI 하나 선택
 - Kafka UI
 - 또는 AKHQ

애플리케이션

- ✓ order-service
- ✓ stock-service
- ✓ payment-service
- ✓ delivery-service

Kafka 개념

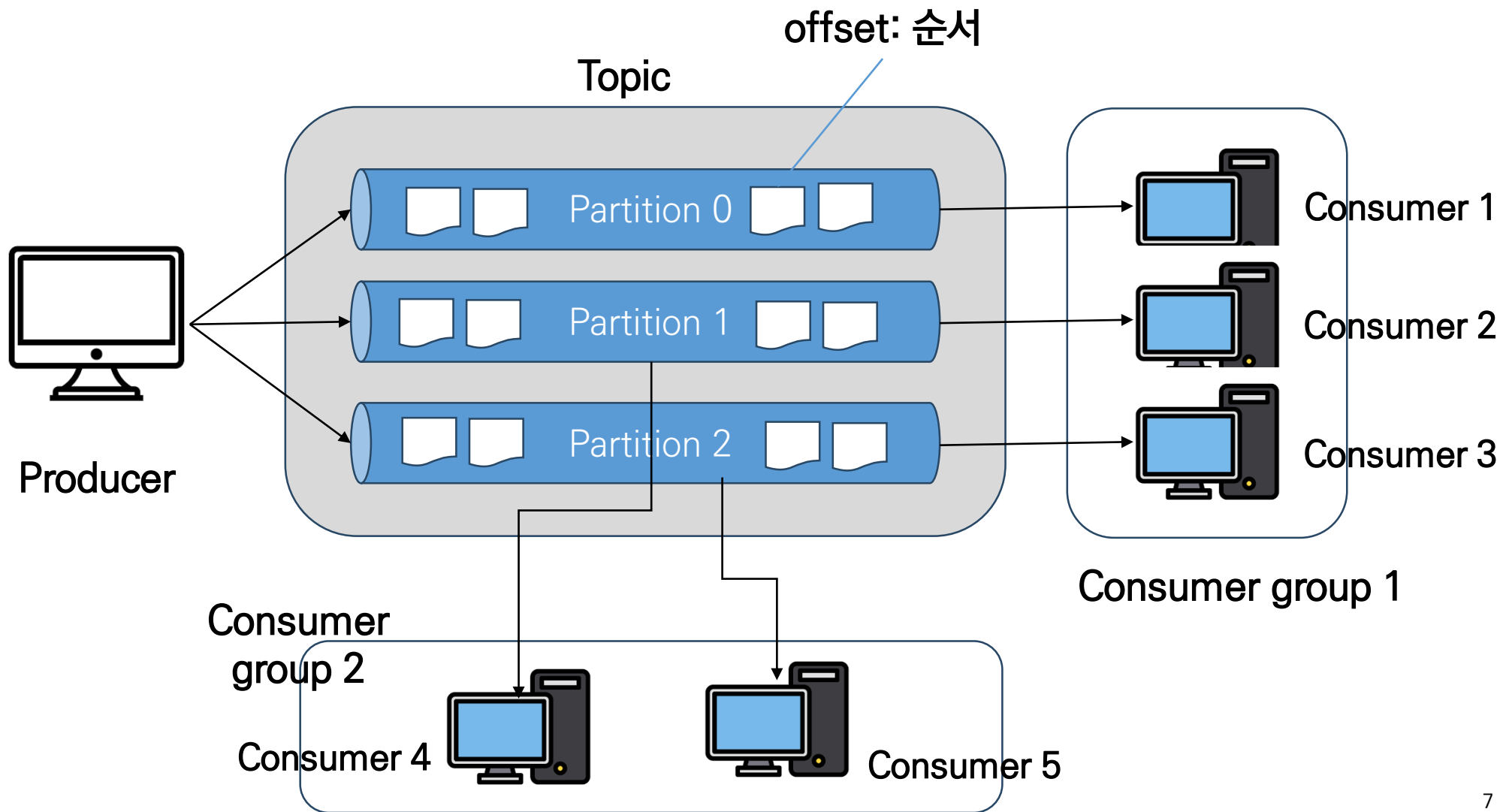
Kafka는 데이터를 “이벤트” 형태로 저장하고, 여러 시스템이 이를 실시간으로 읽고 처리할 수 있게 하는 분산 이벤트 처리 시스템



Kafka 개념

- ✓ Topic (토픽) : 데이터가 들어가는 “논리적 채널”
 - order-events
 - payment-events
- ✓ Producer (생산자) : 데이터를 Kafka에 보내는 주체
 - order-service → order-events
- ✓ Consumer (소비자) : 데이터를 읽는 주체
 - stock-service ← order-events
- ✓ Partition : 토픽 내부 데이터를 분산 저장하는 단위
- ✓ Group: 하나의 서비스를 구성하는 Consumer들의 논리적 집합

Kafka 개념



Kafka 개념

- ✓ 파티션의 수는 시스템 운영자가 결정하며, 컨슈머 객체수(pod)보다 커야 함
- ✓ 하나의 이벤트 흐름을 구분하는 키(혹은 ID)에 따라 파티션이 결정됨
 - 예: OrderCreated → orderId=1001
 - PaymentCompleted → orderId=1001
 - DeliveryStarted → orderId=1001
 - OrderCompleted → orderId=1001

} 같은 파티션 사용
- ✓ 이벤트의 순서를 보장하기 위해 중요

Kafka 개념

- ✓ Kafka의 진짜 핵심은 로그 구조
 - [0] OrderCreated
 - [1] OrderCreated
 - [2] OrderCancelled
 - [3] OrderCreated
- ✓ 특징:
 - 데이터는 append-only (뒤에 계속 추가)
 - 삭제/수정 안 함 (기본적으로)
 - 순서 유지

Kafka 내부 구조

- ✓ Partition
 - 병렬 처리 + 확장성 핵심
 - 각 partition은 순서 보장
 - 서로 다른 partition은 병렬 처리
- ✓ 메시지 저장 방식
 - Partition 0
 - [0] OrderCreated
 - [1] OrderCreated
 - [2] OrderCancelled
 - 각 메시지는 offset (번호) 를 가짐

Topic

└── Partition 0

└── Partition 1

└── Partition 2

Consumer 동작 방식 (핵심)

- ✓ Kafka는 push가 아니라 pull 방식 (처리후에도 이벤트는 남아 있음)
 - 재처리 가능
 - 장애 복구 쉬움
 - 속도 제어 가능
 - 예: 컨슈머가 offset 10부터 읽음

Kafka 중요성

✓ 효과:

- 느슨한 결합 (Loose coupling)
- 비동기 처리
- 확장성
- 장애 격리

SAGA에서 Kafka 역할

OrderCreated → Kafka → Stock
StockReserved → Kafka → Payment
PaymentFailed → Kafka → Stock (보상)

- ✓ 서비스 간 “상태 변화 이벤트”를 전달하는 중심 허브

Kafka의 핵심 특징

- ✓ 고성능:
 - 디스크 기반인데도 빠름 (순차 I/O)
- ✓ 확장성
 - partition 추가로 scale-out
- ✓ 내구성
 - replication 지원
- ✓ 재처리 가능
 - offset 기반
- ✓ 다중 소비 가능
 - 여러 consumer group

Producer → [Kafka Topic (Log)] → Consumer

- 메시지는 사라지지 않음
- offset으로 읽음
- 여러 consumer 가능
- 이벤트 기반 시스템 핵심

Kafka와 NATS 비교

항목	Apache Kafka	NATS
기본 목적	대규모 이벤트 스트리밍 / 로그 저장 / 데이터 파이프라인	초경량 메시징 / 실시간 통신 / 마이크로서비스 연결
저장 방식	디스크 기반 장기 저장	메모리 중심 (JetStream 사용 시 저장 가능)
메시지 보존	강력 (Retention 정책)	기본 짧음, JetStream으로 확장
처리 속도	높음	매우 높음 (초저지연)
순서 보장	파티션 단위 강력	Subject 단위 제한적
Consumer Group	강력 지원	Queue Group 지원
확장성	매우 높음 (대규모 클러스터)	높음 (경량 확장)
장애 복구	강력	단순
메시지 재처리	매우 우수	JetStream 필요
Exactly Once	지원 (복잡)	제한적
Throughput	매우 높음	높음
Kubernetes 환경	가능하지만 무거움	매우 적합
단점	운영 복잡, 리소스 큼	장기 저장/복잡 이벤트 처리 약함

Kafka 설치: Strimzi 사용

Strimzi : Kafka를 Kubernetes 리소스(CRD)로 선언하고, 자동으로 생성·운영·복구까지 해주는 Kafka 전용 Operator

- ✓ Kubernetes 친화적
- ✓ Operator 기반
- ✓ Kafka, Topic, User를 CRD로 관리 가능

Kafka 설치 절차(도커 데스크탑 쿠버네티스)

- ✓ `kubectl create namespace kafka`
- ✓ `kubectl apply -f https://strimzi.io/install/latest?namespace=kafka -n kafka`
- ✓ `kubectl get pod -n kafka --watch`
- ✓ `kubectl apply -f https://strimzi.io/examples/latest/kafka/kafka-single-node.yaml -n kafka`
- ✓ `kubectl wait kafka/my-cluster --for=condition=Ready --timeout=300s -n kafka`

Kafka UI 설치: provectuslabs/kafka-ui:latest

UI for Apache Kafka ⚠ 83b5a60 2024. 4. 10. 18시 41분 25초

Dashboard Topics / order-events

my-cluster ^

Brokers

Topics

Consumers

Overview **Messages** Consumers Settings Statistics

Seek Type **Offset** **Offset** Partitions **All items are selected.** Key Serde **String** Value Serde **String**

	Offset	Partition	Timestamp	Key	Preview
+	0	0	2026. 4. 3. 0시 3분 43초	0efd2cab-3ca4-409e-b254-6a955542de98	
+	1	0	2026. 4. 3. 0시 13분 13초	6fe236fb-f7c5-4687-8cbc-7773353a4054	
+	2	0	2026. 4. 3. 15시 18분 30초	03028395-6c30-4fb2-9506-39a1e6b65118	
+	3	0	2026. 4. 3. 15시 19분 36초	88a50740-13a6-4e08-8419-f15b96f00168	
+	4	0	2026. 4. 3. 15시 19분 58초	28d80ea0-ff0b-48d1-94f6-c795f3806eaf	
+	5	0	2026. 4. 3. 15시 20분 3초	b9968047-087b-426a-8504-edec7c3709f0	
+	6	0	2026. 4. 3. 15시 20분 11초	2661df23-9e15-45fa-a180-8be137a2191f	

Kafka basic 실습

- ✓ 간단한 이벤트 생성 (producer) 및 소비 (consumer) 예제
- ✓ 토픽은 핵심 통로 역할 (order-event)
- ✓ producer 는 이벤트 serialize, consume는 이벤트 desirialize 해야 함
- ✓ Jackson 라이브러리 의존성 포함해야 함

역할	설명
Producer	이벤트 발행
Kafka	이벤트 저장
Consumer	이벤트 처리

Kafka basic 실습

- ✓ 일반 Kafka 이벤트 송수신에는 KafkaConnector가 필요 없음

Order Service

└─ KafkaTemplate → Kafka Topic

Payment Service

└─ @KafkaListener ← Kafka Topic

```
spring:
```

```
  kafka:
```

```
    bootstrap-servers: my-cluster-kafka-bootstrap.kafka:9092
```

Kafka basic 실습

- ✓ CDC는 애플리케이션이 직접 이벤트를 만드는 것이 아니라, DB 변경 로그(binlog)를 읽어서 Kafka 이벤트로 바꾸는 구조
- ✓ KafkaConnector는 Strimzi에서 Debezium Connector 설정을 Kubernetes 리소스로 관리하기 위한 것

구분	일반 Kafka 이벤트	CDC
이벤트 생성 주체	애플리케이션 코드	Debezium Connector
KafkaConnector 필요	불필요	필요
데이터 출처	서비스 로직	DB 변경 로그
사용 예	OrderCreated 발행	orders 테이블 INSERT 감지
구현 방식	KafkaTemplate, @KafkaListener	Kafka Connect + Debezium

Kafka basic 실습

← → ↺ 🏠 ⓘ localhost:30080/index.html ☆ 👤 ⋮

🍷 | 🍷 쿠팡!



Kafka Event Monitor

Real-time Producer & Consumer Event Tracking

Create Order Event

Product ID

PROD-001

Quantity

1

Amount

100

Send Event

Clear All

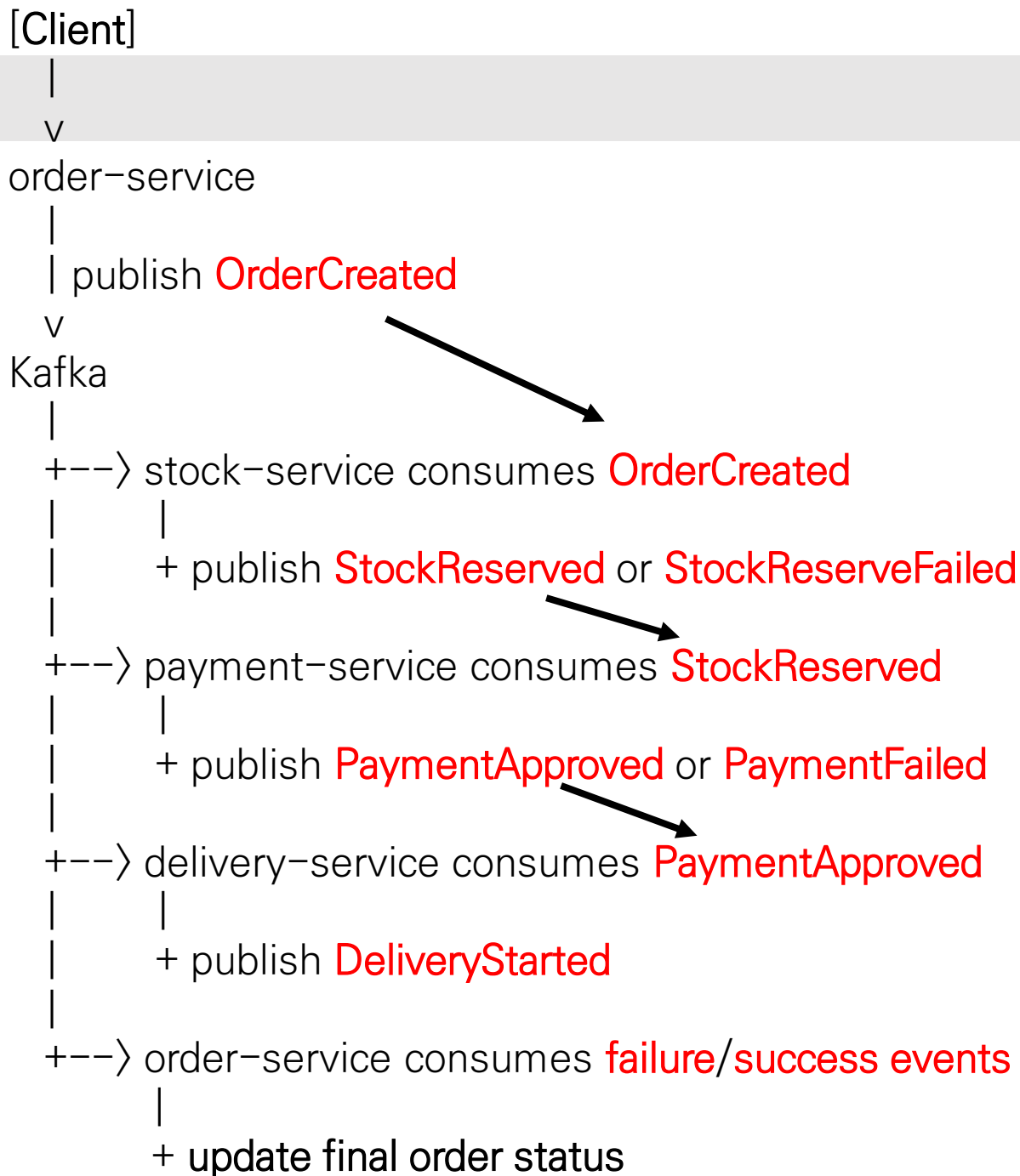
● Producer Events 6

Event Type	Order ID	Product ID	Qty	Amount
OrderCreated	9f382691-9a09-453d...	PROD-001	1	100
OrderCreated	9e2540ba-50d1-4f41...	PROD-001	1	100

● Consumer Events 6

Event Type	Order ID	Product ID	Qty	Amount
OrderCreated	9f382691-9a09-453d...	PROD-001	1	100
OrderCreated	9e2540ba-50d1-4f41...	PROD-001	1	100

실습 이벤트 흐름



주문 상태

- ✓ CREATED
- ✓ STOCK_CONFIRMED
- ✓ PAYMENT_CONFIRMED
- ✓ COMPLETED
- ✓ CANCELLED

실습 시나리오 1: 재고 부족

1. 주문 생성
2. OrderCreated 발행
3. 재고 서비스가 수량 확인
4. 재고 부족이면 StockReserveFailed
5. 주문 서비스가 받아서 CANCELLED

실습 시나리오 2: 결제 실패

1. 주문 생성
2. 재고 확보 성공 → StockReserved
3. 결제 실패 → PaymentFailed
4. 재고 서비스가 PaymentFailed 받아서 StockReleased
5. 주문 서비스는 최종 CANCELLED

실습 시나리오 3: 모두 성공

1. OrderCreated
2. StockReserved
3. PaymentApproved
4. DeliveryStarted
5. 주문 상태 COMPLETED

토픽 설계

- ✓ order-events
- ✓ stock-events
- ✓ payment-events

서비스별 토픽 분리

- ✓ order-service → order-events
- ✓ stock-service → stock-events
- ✓ payment-service → payment-events

이벤트 메시지 구조 예시

```
{
  "eventId": "uuid",
  "eventType": "OrderCreated",
  "sagaId": "uuid",
  "orderId": "1001",
  "timestamp": "2026-03-31T18:00:00",
  "payload": {
    "productId": "P001",
    "quantity": 2,
    "amount": 30000
  }
}
```

SAGA 구현 방식

- ✓ Choreography
 - 각 서비스가 이벤트를 보고 다음 행동을 수행
 - 중앙 조정자가 없음
- ✓ Orchestration
 - 중앙 오케스트레이터가 각 서비스를 지휘

서비스별 역할

✓ order-service

- 주문 생성 API 제공
- 주문 저장
- OrderCreated 발행
- 성공/실패 이벤트 소비
- 최종 상태 갱신

✓ payment-service

- StockReserved 구독
- 결제 승인/실패 시뮬레이션성공 시 PaymentApproved
- 실패 시 PaymentFailed

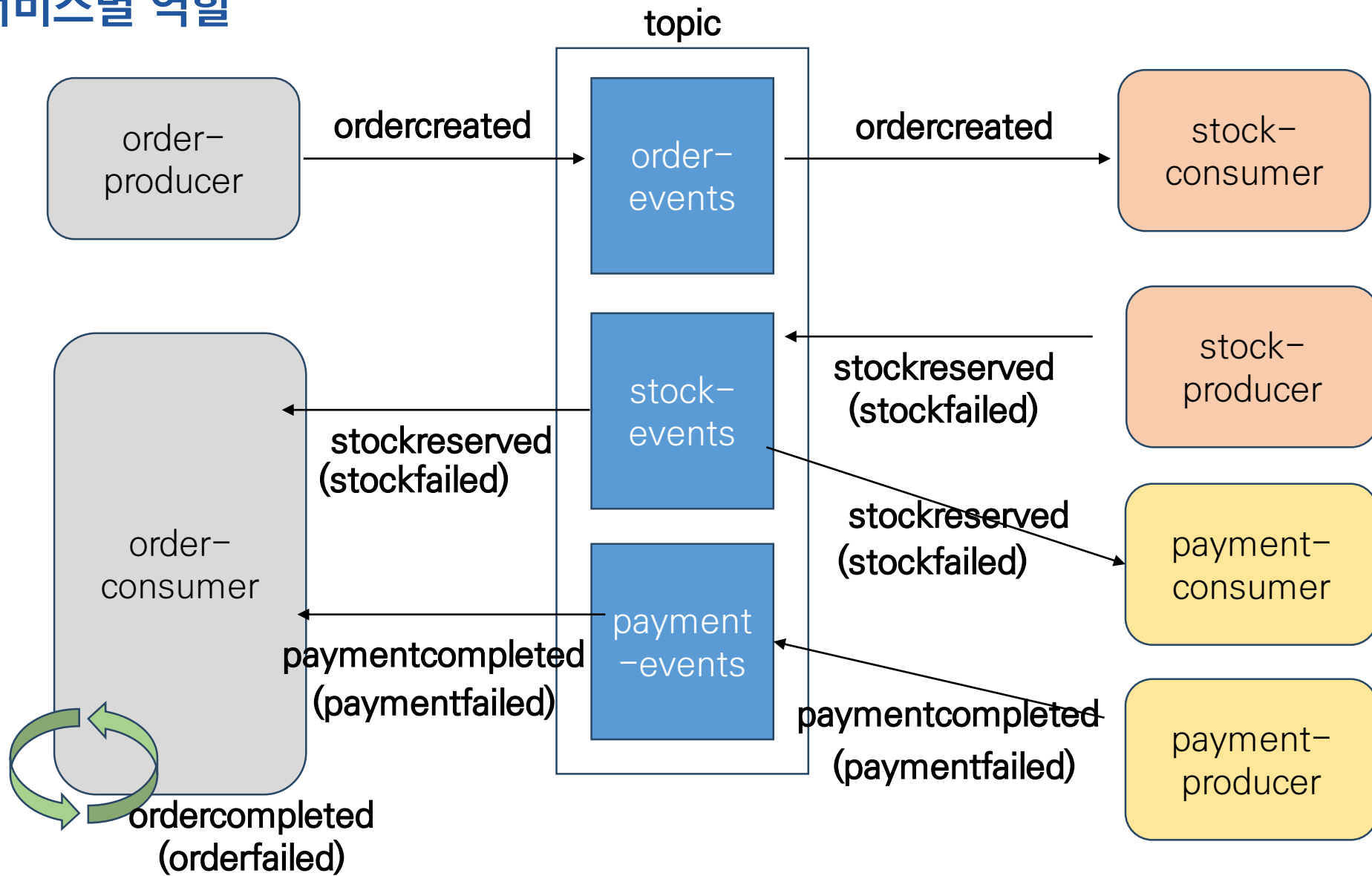
✓ stock-service

- OrderCreated 구독
- 재고 확인 및 차감성공 시 StockReserved
- 실패 시 StockReserveFailed
- 결제 실패 시 StockReleased 처리

✓ delivery-service

- PaymentApproved 구독
- 배송 생성
- DeliveryStarted 발행

서비스별 역할



실패 시뮬레이션 방법

- ✓ 요청값으로 실패를 유도
 - 예:상품 수량이 100 이상이면 재고 실패
 - 금액이 50000 이상이면 결제 실패
 - 특정 상품코드면 배송 실패

실습 로드맵

- ✓ 1단계: Kafka만 설치
 - Strimzi 설치
 - Kafka single-node 생성
 - 토픽 생성
 - producer/consumer 테스트
- ✓ 2단계: order-service + stock-service
 - 주문 생성
 - 주문 이벤트 발행
 - 재고 서비스 소비
 - 성공/실패 이벤트 회신

실습 로드맵

- ✓ 3단계: payment-service 추가
 - StockReserved 이후 결제 처리
 - 실패 시 보상 이벤트
- ✓ 4단계: 관찰 포인트 추가
 - Kafka UI
 - 로그
 - 상태 테이블
 - sagald 추적

실습 1: 주문 성공 케이스

← → ↺ 🏠 ⓘ localhost:30300/index.html 🔍 ☆ 📁 👤

🍷 새 탭 8 Container Orches... ISO Committee ISO/TC... ISO ISO/TC 307 - Block... <> 쿠버네티스 시작하... ☁ 클라우드스토어 씨... 🌐 Home Page - Clou... >> 📁 모든 !

Order Saga

주문 정보를 웹에서 등록하면 MySQL에 저장됩니다.

상품 ID

p100

재고 확인

현재 재고: 4개

수량

1

금액

1000

주문 생성

이벤트 로그

Kafka로 들어온 이벤트/주문 서비스 이벤트를 표시합니다. (5초 자동 갱신)

시간	소스	이벤트	주문ID	비고
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockReleased	303a6eac-1246-4d55-88a2-8bcb3e6232ff	
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCancelled	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCancelled	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockFailed	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockFailed	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCreated	303a6eac-1246-4d55-88a2-8bcb3e6232ff	

실습 2: 재고 부족 케이스

←

→

↺

🏠

🔍

localhost:30300/index.html

☆

🔖

👤

🍷

새 탭

8 Container Orches...

Committee ISO/TC...

ISO/TC 307 - Block...

쿠버네티스 시작하...

클라우드스토어 씨...

Home Page - Clou...

>>

모든

Order Saga

주문 정보를 웹에서 등록하면 MySQL에 저장됩니다.

상품 ID

p100

재고 확인

현재 재고: 4개

수량

5

금액

1000

주문 생성

이벤트 로그

Kafka로 들어온 이벤트/주문 서비스 이벤트를 표시합니다. (5초 자동 갱신)

시간	소스	이벤트	주문ID	비고
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockReleased	303a6eac-1246-4d55-88a2-8bcb3e6232ff	
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCancelled	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	ORDER_SERVICE	OrderCancelled	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockFailed	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후 1:29:34	STOCK_SERVICE	StockFailed	303a6eac-1246-4d55-88a2-8bcb3e6232ff	재고 부족
2026. 4. 4. 오후	ORDER_SERVICE	OrderCreated	303a6eac-1246-4d55-88a2-	

실습 3: 결제 실패 케이스 -> amount가 50000보다 크면 실패

←

→

↺

🏠

🔍

☆

📁

👤

localhost:30300/index.html

🔗 새 탭

📁 8 Container Orches...

📁 Committee ISO/TC...

📁 ISO/TC 307 - Block...

📁 쿠버네티스 시작하...

📁 클라우드스토어 씨...

📁 Home Page - Clou...

>> 📁 모든

Order Saga

주문 정보를 웹에서 등록하면 MySQL에 저장됩니다.

상품 ID

p100

재고 확인

현재 재고: 4개

수량

1

금액

60000

주문 생성

주문 생성 완료: f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc

이벤트 로그

Kafka로 들어온 이벤트/주문 서비스 이벤트를 표시합니다. (5초 자동 갱신)

시간	소스	이벤트	주문ID	비고
2026. 4. 4. 오후 1:41:36	STOCK_SERVICE	StockReleased	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	
2026. 4. 4. 오후 1:41:36	ORDER_SERVICE	OrderCancelled	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	결제 승인 실패
2026. 4. 4. 오후 1:41:36	ORDER_SERVICE	OrderCancelled	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	결제 승인 실패
2026. 4. 4. 오후 1:41:36	ORDER_SERVICE	StockReserved	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	
2026. 4. 4. 오후 1:41:36	PAYMENT_SERVICE	PaymentFailed	f387aa1e-2db3-4a90-9a3f-bdcfa56ef9fc	결제 승인 실패
2026. 4. 4. 오후	STOCK_SERVICE	StockReserved	f387aa1e-2db3-4a90-9a3f-	

실습 4: 재고량 조정

→ ↺ 🏠 ⓘ localhost:30302/index.html 🔍 ☆ 📁 👤

🔍 새 탭 🍷 8 Container Orches... 📄 Committee ISO/TC... 📄 ISO/TC 307 - Block... 📄 쿠버네티스 시작하... ☁️ 클라우드스토어 씨... 📄 Home Page - Clou... >> 📁 모든

Stock Saga Dashboard

재고와 Saga 이벤트 상태를 실시간으로 확인합니다.

새로고침

재고 현황

상품 수: 4 마지막 변경: 2026-04-04T13:41:36.531102

상품ID	수량	업데이트
P100	4	2026-04-04T13:41:36.531102
P200	1	2026-04-04T03:25:11.365849
p300	6	2026-04-04T13:29:34.348531
p400	21	2026-04-04T13:14:26.64778

재고 수량 등록/수정

저장

주문(OrderCreated) 이벤트가 오면 해당 상품 재고가 자동 차감됩니다.

이벤트 상태 로그 (최신 200건)

로그 수: 127 최신 이벤트: 2026-04-04T13:41:36.544962

동기 호출방식과 차이

✓ REST 체인:

- 앞 서비스가 뒤 서비스 응답을 기다림
- 장애 전파 쉬움
- 결합도 높음

✓ 이벤트 기반:

- 발행 후 비동기 처리
- 느슨한 결합
- 최종 일관성(eventual consistency)
- 보상 트랜잭션 필요

CQRS: Command Query Responsibility Segregation

- ✓ 쓰기(Command)와 읽기(Query)를 분리하는 아키텍처 패턴
- ✓ Command (쓰기) → 상태 변경 → 이벤트 발생
- ✓ Query (조회) → 조회 전용 모델 → 빠른 응답
- ✓ 쓰기
 - order-service → OrderCreated (Kafka)
 - stock-service → 재고 차감
 - payment-service → 결제
- ✓ 읽기
 - order-service → Feign → stock-service

CQRS 이유

✓ 성능

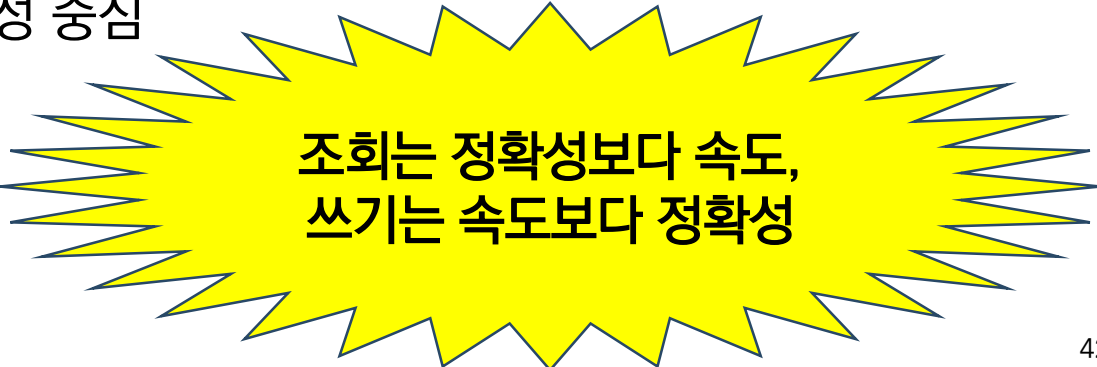
- 조회와 쓰기를 분리하면 조회는 빠르게 (캐시, read replica) 쓰기는 안전하게 (트랜잭션, 이벤트)

✓ 복잡성 분리

- 쓰기: 복잡 (재고, 결제, 보상) 조회: 단순 (값 보여주기)

✓ 확장성

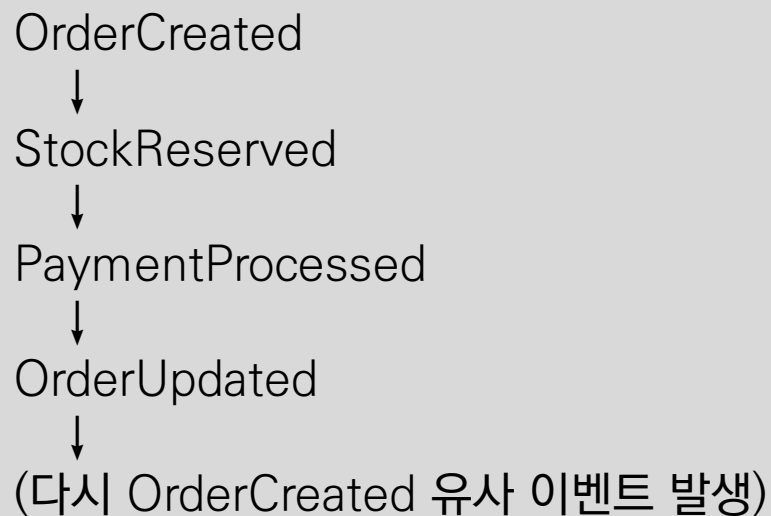
- 조회 트래픽 많음 → 따로 스케일
- 쓰기 트래픽 적음 → 안정성 중심



조회는 정확성보다 속도,
쓰기는 속도보다 정확성

유의 사항: 이벤트 루프

- ✓ 이벤트 잘못 설계시 이벤트가 루프를 형성하면 메시지 폭증 하는 현상
- ✓ 사례:



유의 사항: 이벤트 루프

✓ 발생 패턴

원인	설명
이벤트 재발행 조건 오류	상태 변경마다 동일 이벤트 발행
Saga 보상 로직 오류	실패→보상→재시도 반복
중복 소비	Idempotency 부족
여러 서비스 간 상호 트리거	$A \rightarrow B \rightarrow C \rightarrow A$
DLQ 재처리 실수	실패 이벤트 반복

이벤트 루프 방지 방안

- ✓ 이벤트 타입 명확화
 - 예: OrderCreated
OrderConfirmed
OrderCancelled
 - 동일 이벤트 재발행 금지
- ✓ Topic 역할 분리
 - commands-topic
 - events-topic
 - compensation-topic

이벤트 루프 방지 방안

✓ Idempotency

- eventId 저장
- 중복 처리 방지

✓ Saga 상태 관리

- sagaId 추적
- 단계별 상태 저장

✓ TTL / Retry 제한

- 최대 재시도 횟수
- DLQ 활용

- TTL (Time To Live) : 메시지, 데이터, 캐시, 리소스가 유효한 최대 생존 시간
- DLQ (Dead Letter Queue / Dead Letter Topic): 정상 처리에 실패한 메시지를 별도로 격리 저장하는 공간

성공 시나리오

1. 주문 생성
2. Order Service → OrderCreated
3. Orchestrator → Stock Service에 ReserveStockCommand
4. Stock 성공 → StockReserved
5. Orchestrator → Payment Service에 ProcessPaymentCommand
6. Payment 성공 → PaymentCompleted
7. Orchestrator → Order Service에 CompleteOrderCommand
8. 주문 완료

실패 시나리오 1: 재고 부족

1. 주문 생성
2. Stock 실패 → StockFailed
3. Orchestrator → Order Service에 FailOrderCommand
4. 주문 실패

실패 시나리오 2: 결제 실패

1. 주문 생성
2. Stock 성공
3. Payment 실패 → PaymentFailed
4. Orchestrator → Stock Service에 ReleaseStockCommand
5. 재고 복구
6. Orchestrator → Order Service에 FailOrderCommand
7. 주문 실패

서비스

- ✓ order-service
- ✓ stock-service
- ✓ payment-service
- ✓ orchestrator-service

토픽

- ✓ order-events
- ✓ stock-commands
- ✓ stock-events
- ✓ payment-commands
- ✓ payment-events
- ✓ order-commands

